МОДЕЛИРОВАНИЕ ПРОГРАММНОЙ СИСТЕМЫ В НОТАЦИИ UML

Методические указания к практической работе №4

1. Цель и результат работы

Цель: построить целостную модель проектируемой программной системы в нотации UML, связывающую требования (спецификация требований, схема данных и спецификация данных предыдущих этапов) с архитектурой и поведением системы.

Результат этапа – пакет документов:

- комплект из 5–8 диаграмм UML. Обязательные: диаграмма прецедентов, диаграмма классов, диаграмма последовательности, диаграмма деятельности. Рекомендуемые (приносят дополнительные баллы и полноту модели): диаграмма состояний, диаграмма компонентов, диаграмма развертывания, диаграмма сущность–связь (ER-диаграмма);
- пояснительная записка: для каждой диаграммы указываются её назначение в проекте, какие требования она покрывает (ссылки на идентификаторы ПТ/БТ/СХ/О/АК), принятые допущения и ограничения моделирования;
- матрица трассировки «требование → диаграмма → элемент модели»;
- исходники диаграмм (Mermaid-код либо файлы проекта draw.io / StarUML) и экспортированные изображения.

Главное правило этапа: диаграммы не создаются независимо друг от друга. Модель оценивается не по красоте картинок, а по согласованности: классы с диаграммы классов должны фигурировать в диаграммах последовательности, прецеденты – реализовываться потоками деятельности, а актёры должны совпадать во всех диаграммах (имена – буквально до буквы).

2. Распределение работ по ролям

Роль	**Зона ответственности**	**Конкретные задачи на этапе моделирования**
Тимлид	«Сойдётся ли модель в целом»	Проверка сквозной согласованности всех диаграмм (одни и те же сущности, одинаковые имена, непротиворечивые потоки); приёмка финальной версии; подготовка презентации
Владелец продукта	«Что моделируем»	Формулирование актёров и приоритетов прецедентов; выбор 2–3 ключевых сценариев для детальной проработки; валидация модели с точки зрения требований; правка текстов сценариев
Менеджер проекта	«Как и когда моделируем»	План работ с датами по каждой диаграмме; реестр диаграмм и статусы готовности; организация двух внутренних ревью; сборка матрицы трассировки

Программисты	«Как система устроена»	Разработка диаграмм классов, последовательности, состояний, деятельности, компонентов, развертывания; техническая согласованность (имена классов, сигнатуры операций)

Диаграмма прецедентов делается совместно: владелец продукта задаёт бизнес-сценарии, программисты переводят их в нотацию UML, тимлид контролирует полноту покрытия требований.

3. Инструмент моделирования: Mermaid

Диаграммы выполняются текстовым описанием на языке Mermaid. Преимущества для командной работы: диаграмма хранится как обычный текст (удобно версионировать в Git и ревьюить), изменения вносятся правкой строк, а не перетаскиванием мышью.

- Онлайн-редактор: mermaid.live – пишете код, сразу видите картинку, экспортируете PNG/SVG.
- Mermaid-код вкладывается прямо в Markdown-файлы (GitHub, GitLab, Obsidian, VS Code с расширением Mermaid).
- Альтернативы, если команда предпочитает: draw.io, PlantUML, StarUML. Требования к содержанию диаграмм одинаковы для любого инструмента.

4. Требования к отдельным диаграммам

4.1. Диаграмма прецедентов (Use Case Diagram)

Назначение: показать функциональные границы системы – кто и что может делать.

Обязательные элементы: актёры; прецеденты; ассоциации «актёр – прецедент»; отношения include и extend (не менее двух); системная граница (System Boundary).

Методика построения:

- выписать всех участников из спецификации требований: люди, внешние системы, устройства. Актёр – это роль, а не конкретный человек и не «пользователь» вообще;
- формулировать прецеденты глаголами результата («сгенерировать движение», «проверить выполнение движений»), а не отглагольными существительными («работа с движениями» – ошибка);
- крупные прецеденты декомпозировать: например, «Создать хореографию» включает «Сохранить хореографию» (include); «Отменить запрос генерации» расширяет основной сценарий (extend);
- для каждого ключевого прецедента заполнить шаблон описания (см. таблицу ниже).

Поле	**Содержание**
ID и название	UC-9 «Запросить генерацию движения»
Актёры	основной и второстепенные
Предусловия	что должно быть истинно до начала сценария

Основной поток	пронумерованные шаги «актёр → система → актёр»
Альтернативы и исключения	не менее двух веток на ключевой прецедент
Постусловия	состояние системы после завершения
Трассировка	ссылки на требования (ПТ, БТ, СХ и т. д.)

Контрольные цифры: 8–15 прецедентов, 2–4 актёра, полные описания не менее чем для трёх ключевых прецедентов.

4.2. Диаграмма классов (Class Diagram)

Назначение: статическая структура системы – сущности, атрибуты, операции и связи. Это развитие и уточнение схемы данных, построенной на предыдущем этапе: теперь появляется не только данные, но и поведение.

Правила построения:

- классы – существительные из предметной области и требований (выпишите ключевые существительные из пользовательских требований – это заготовки классов);
- атрибуты и операции записываются в нотации UML с указанием видимости: + public, − private;
- различать типы связей осознанно: ассоциация, агрегация (пустой ромб), композиция (закрашенный ромб), наследование (пустая стрелка), зависимость;
- каждая ассоциация получает имя и кратность (1, 0..1, 1..*, 0..*);
- не менее двух классов со стереотипами: <<entity>>, <<control>>, <<boundary>> или прикладными – <<service>>, <<repository>>.

Контрольные цифры: 8–12 классов, атрибуты с типами данных, операции, все связи с кратностями; минимум одна композиция и одно наследование – если они действительно есть в предметной области (не рисуйте наследование «для галочки»).

4.3. Диаграмма последовательности (Sequence Diagram)

Назначение: показать во времени взаимодействие объектов при реализации ключевого прецедента. Это «динамическая проекция» диаграммы классов.

- моделируются 2–3 ключевых сценария – их выбирает владелец продукта. Критерий выбора: высокая ценность для пользователя + высокая архитектурная рискованность;
- линии жизни – экземпляры классов с диаграммы классов; имена обязаны совпадать;
- сообщения соответствуют операциям классов; асинхронные сообщения (для долгих операций) отличать от синхронных;
- использовать фрагменты: alt (альтернативы), opt (опциональные шаги), loop (повторы), par (параллельность);
- показывать возвраты результатов (пунктирные стрелки) и примечания Notes с привязкой к требованиям.

4.4. Диаграмма деятельности (Activity Diagram)

Назначение: логика сквозного процесса – что происходит, в какой последовательности, где система ждёт решения человека или внешнего события.

- использовать дорожки (swimlanes) по актёрам или подсистемам – это сразу показывает, где локализована ответственность. В Mermaid swimlanes не поддерживаются, поэтому дорожки передаются секциями subgraph с указанием исполнителя;
- развилки оформляются ромбами с условиями на исходящих стрелках; у процесса ровно один начальный узел и хотя бы один конечный;
- минимум один параллельный поток (fork/join) – там, где операции действительно независимы;
- долгие операции показывать сигналами ожидания (асинхронные события), а не «висящим» действием.

4.5. Диаграмма состояний (State Machine Diagram)

Назначение: жизненный цикл одной ключевой сущности – из каких состояний она состоит и под действием каких событий переходит из состояния в состояние.

- выбирается сущность с реальной динамикой (например, запрос на генерацию, заказ, документ). Статичный справочник (например, «Стиль танца») – плохой выбор;
- каждая дуга перехода подписывается в формате: событие [условие] / действие;
- обязательно показать ветки ошибки и отмены: «happy path без отклонений» считается незавершённой моделью;
- состояния должны соответствовать статусам, описанным в требованиях.

4.6. Диаграмма компонентов (Component Diagram)

Назначение: архитектура системы – крупные функциональные части и их контракты (интерфейсы).

- компоненты выделяются по функциональным подсистемам («Модуль генерации», «Каталог движений»), а не механически «слой 1, слой 2»;
- показываются предоставляемые интерфейсы (lollipop) и требуемые (socket);
- не менее 5 компонентов, включая внешне-интеграционный (датчики, внешние сервисы);
- компоненты должны опираться на классы из диаграммы классов – сохраняется прослеживаемость модели.

4.7. Диаграмма развертывания (Deployment Diagram)

Назначение: физическое размещение – на каких узлах что исполняется и как узлы связаны.

- узлы – реальные вычислительные устройства: клиентские, серверные, оборудование;
- внутри узлов размещаются артефакты/компоненты, соответствующие диаграмме компонентов;

- на линиях связи указываются протоколы (HTTPS, WebSocket, gRPC, USB-OTG, Bluetooth и т. п.);
- если требования фиксируют оборудование или платформы (камеры, датчики, ОС) – они обязаны быть отражены: это прямая трассировка на системные характеристики.

5. Шаблоны Mermaid для каждого типа диаграмм

Ниже – минимальные заготовки синтаксиса для кейса «ИИ-хореограф». Их нужно адаптировать под свою предметную область: копировать как есть – не эффективно.

5.1. Диаграмма прецедентов (через flowchart)

В Mermaid нет отдельного синтаксиса прецедентов, поэтому используется блок-схема flowchart: актёры – узлы-круги за пределами рамки, прецеденты – узлы-скругления внутри subgraph (системная граница), include/extend – пунктирные связи.

```
flowchart LR
    Guest(["Актёр: Гость"])
    Dancer(["Актёр: Хореограф"])

    subgraph System["Система"]
        UC1(["UC-1 Зарегистрировать аккаунт"])
        UC2(["UC-2 Авторизоваться"])
        UC3(["UC-3 Просмотреть каталог"])
        UC4(["UC-4 Найти по фильтрам"])
        UC5(["UC-5 Создать хореографию"])
        UC8(["UC-8 Сохранить хореографию"])
    end

    Guest --> UC1
    Guest --> UC3
    Dancer --> UC5
    UC4 -.extend.-> UC3
    UC5 -.include.-> UC8
```

5.2. Диаграмма классов (classDiagram)

```
classDiagram
    class User {
        <<entity>>
        -id : UUID
        -email : String
        +register() void
        +login() void
    }
    class Choreography {
        <<entity>>
        -id : UUID
        -title : String
        +save() void
    }
    User "1" *-- "0..*" Choreography : владеет
```

Виды связей в classDiagram: *-- композиция, o-- агрегация, -- ассоциация, --|> наследование, ..> зависимость.

5.3. Диаграмма последовательности (sequenceDiagram)

```
sequenceDiagram
    autonumber
    actor D as Хореограф
    participant UI as UI приложения
    participant C as Контроллер
    D->>UI: задатьПараметры(...)
    UI->>C: создатьЗапрос(params)
    C--)G: generate(params)   %% долгая операция – асинхронно
    alt движение готово
        G--)C: результат
    else пользователь отменил
        C->>G: cancel()
    end
```

Ключевые конструкции: ->> синхронное сообщение; --) асинхронное; -->> возврат результата; alt/else альтернативы; loop повторы; par параллельные ветки; Note over примечания.

5.4. Диаграмма деятельности (через flowchart TD)

Дорожки передаются секциями subgraph с указанием исполнителя; развилка – ромб {условие?}.

```
flowchart TD
    Start([Начало]) --> Check{Устройство совместимо?}
    Check -- нет --> Err[Сообщение об ошибке] --> End([Конец])
    Check -- да --> Calib[Провести калибровку]
    Calib --> Compare[Сравнить с эталоном]
    Compare --> Dec{Решение пользователя}
    Dec -- принять --> Save[Сохранить отчёт] --> End
    Dec -- повторить --> Calib
```

5.5. Диаграмма состояний (stateDiagram-v2)

Имена состояний с пробелами оформляются как state "Имя с пробелами" as id.

```
stateDiagram-v2
    state "В очереди" as queued
    state "Генерация" as generating
    [*] --> queued : отправлен в очередь
    queued --> generating : взято в обработку
    generating --> queued : ошибка / повторить
    generating --> [*] : результат сохранён
```

5.6. Диаграмма компонентов (graph TB с подписями интерфейсов)

```
graph TB
    subgraph Client["Клиентская часть"]
        UI[Приложение: UI и 3D-рендер]
    end
    subgraph Server["Серверная часть"]
        Auth[Модуль аутентификации]
        Gen[Модуль генерации]
    end
```

```
    UI -- "предоставляет IAccountManagement (HTTPS)" --> Auth
    UI -- "требует IGeneration (gRPC)" --> Gen
```

5.7. Диаграмма развертывания (graph TB, узлы = подграфы)

```
graph TB
    subgraph Node1["Узел: Мобильное устройство"]
        App["Клиентское приложение"]
    end
    subgraph Node2["Узел: Сервер приложений"]
        API["API-шлюз"]
    end
    subgraph Node3["Узел: ИИ-сервер (GPU)"]
        Infer["Инференс модели"]
    end
    App -- "HTTPS / WebSocket" --> API
    API -- "gRPC" --> Infer
```

5.8. ER-диаграмма (erDiagram) – связка со схемой данных этапа 2

```
erDiagram
    USER ||--o{ CHOREOGRAPHY : владеет
    CHOREOGRAPHY }o--o{ MOVEMENT : состоит из
    USER {
        uuid id
        string email
    }
```

6. Порядок выполнения работы

Шаг	Содержание	Исполнитель	Результат
1	Инвентаризация требований: выписать актёров, сущности, ключевые процессы	Вся команда	Список кандидатов для моделирования
2	Выбор 2–3 ключевых сценариев для детальной проработки	Владелец продукта	Зафиксированный список сценариев
3	Диаграмма прецедентов + описания прецедентов по шаблону	Владелец + программисты	UC-диаграмма, шаблоны описаний
4	Черновик диаграммы классов (уточнение схемы данных)	Программисты	Диаграмма классов v1
5	Диаграммы последовательности по выбранным сценариям	Программисты	2–3 диаграммы
6	Диаграмма деятельности по одному сквозному процессу	Программисты	Диаграмма с дорожками
7	Диаграмма состояний ключевой сущности	Программисты	Диаграмма
8	Ревью: проверка согласованности модели, матрица трассировки	Тимлид + менеджер	Протокол ревью, список правок
9	Диаграммы компонентов и развертывания	Программисты	Две диаграммы
10	Финальная сборка: записка, матрица трассировки, презентация	Тимлид	Готовый пакет документов

7. Типичные ошибки (проверяются при ревью)

- прецедент сформулирован как «работа с…» вместо результата («создать…», «просмотреть…»);
- классы без операций или с операциями, которые нигде не вызываются в диаграммах последовательности;
- имена классов на разных диаграммах не совпадают («User» на одной диаграмме, «Пользователь» на другой);
- диаграмма состояний без ветки ошибки/отмены;
- компоненты без интерфейсов – «прямоугольники со стрелками»;
- диаграммы не привязаны к требованиям: невозможно сказать, какое ПТ или БТ реализует данный элемент;
- на диаграмме деятельности параллельность нарисована там, где в требованиях её нет;
- Mermaid-код не проверен в редакторе: синтаксическая ошибка делает диаграмму неотрисовываемой.

8. Критерии оценки

Критерий	**Что проверяется**	**Вес**
Полнота	Состав диаграмм (5–8), покрытие всех разделов требований (БТ, СХ, ПТ, О, АК)	15%
Согласованность	Единство имён, связность диаграмм между собой, отсутствие противоречий	20%
Корректность нотации	Правильное использование элементов UML: типы связей, кратности, фрагменты alt/loop/par	15%
Трассировка	Матрица «требование → элемент модели», ссылки в примечаниях на диаграммах	15%
Качество сценариев	Наличие альтернатив и исключений, веток ошибок и отмены	15%
Оформление и защита	Записка, экспортированные изображения, ответы на вопросы	20%

9. Рекомендуемая литература и справочники

- Галиаскаров Э.Г. Анализ и проектирование систем с использованием UML : учебник для вузов / Э. Г. Галиаскаров, А. С. Воробьев. – Москва : Издательство Юрайт, 2025. – 125 с. – URL: https://urait.ru/bcode/568178
- Черткова Е.А. Программная инженерия. Визуальное моделирование программных систем : учебник для вузов / Е. А. Черткова. – 2-е изд., испр. и доп. – Москва : Издательство Юрайт, 2020. – 147 с. – URL: https://urait.ru/bcode/452749 .
- Якобсон А., Буч Г., Рамбо Дж. Унифицированный процесс разработки программного обеспечения. - СПб.: Питер, 2002. – 496 с.
- Фаулер М. «UML. Основы. Краткий справочник» – компактная шпаргалка по всем типам диаграмм.
- Официальная документация Mermaid: mermaid.js.org – разделы Class diagrams, Sequence diagrams, State diagrams.